



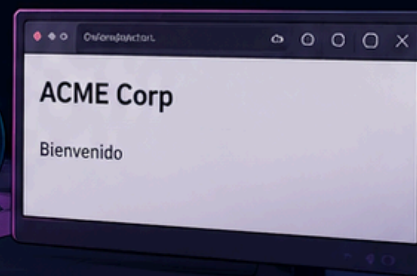
ACME CORP
— INNOVATING THE FUTURE

MEMORIA DEL LABORATORIO DE SEGURIDAD INFORMÁTICA

AUDITORÍA Y ANÁLISIS DE APLICACIONES WEB

OBJETIVO:

Identificación de vulnerabilidades en la infraestructura y aplicación web de ACME Corp mediante técnicas de reconocimiento, enumeración y análisis.



RECONOCIMIENTO
Y ENUMERACIÓN



ANÁLISIS DE
SUBDOMINIOS



FUZZING Y
DESCUBRIMIENTO



HALLAZGOS Y
VULNERABILIDADES



AUTORA:
Paloma



CURSO:
Seguridad Informática

ACME
CORP



ENTORNO:
Laboratorio Local (Docker + NGINX)



METODOLOGÍA:
OWASP Testing Guide

ÍNDICE

1. Levantar el laboratorio.....	2
<!-- TODO: migrate dev.lab.local to production →.....	9
Eso ya es un hallazgo de información sensible.	
Porque te está revelando:.....	9
• existe un entorno dev.....	9
• probablemente separado de producción.....	9
• y puede tener menos seguridad.....	9
• debug activado.....	9
• backups.....	9
• versiones antiguas.....	9
• APIs internas.....	9
Ahora pensamos como atacante.....	9
Tu siguiente pregunta natural sería:.....	9
“¿Qué más hay en dev.lab.local?”.....	9
Así que ahora volvemos a enumerar rutas sobre DEV.....	9
Pensemos:.....	9
• ya sabemos usar ffuf.....	9
• ya sabes que FUZZ sustituye palabras del diccionario.....	9
• ya conoces una wordlist de Web-Content.....	9
Y tenemos que apuntar a:.....	10
http://dev.lab.local:4444/FUZZ.....	10
A partir de ahí:.....	10
• mirar qué devuelve 200.....	10
• qué devuelve 301.....	10
• qué devuelve 403.....	10
• qué nombres parecen interesantes.....	10
Y cuando encontremos algo raro tipo:.....	10
• backups.....	10

• old.....	10
• debug.....	10
• logs.....	10
• zip.....	10
• sql.....	10
Buscar archivos backup.....	11
ffuf -u http://dev.lab.local:4444/FUZZ \	11
-w ~/SecLists/Discovery/Web-Content/common.txt \	11
-e .bak,.old,.zip,.sql,.txt,.env	11
Buscar extensiones típicas.....	11
ffuf -u http://dev.lab.local:4444/FUZZ \	12
-w ~/SecLists/Discovery/Web-Content/common.txt \	12
-e .bak,.old,.zip,.sql,.txt,.env	12
NUCLEI → buscar vulnerabilidades conocidas	12
nuclei -update-templates	12
Escaneo básico.....	12
nuclei -u http://www.lab.local:4444	12
Sí, para el lab te renta tener el stack típico instalado:	13
• ffuf → fuzzing de rutas	13
• httpx → comprobar hosts vivos	13
• nuclei → detectar vulnerabilidades/misconfigs	13
• subfinder → subdominios reales	13
• opcional: katana, waybackurls, hakrawler	13
En Ubuntu hacemos esto para instalar las herramientas nuclei, https, subfinder:....	13
sudo apt update	13
sudo apt install ffuf golang git -y	13
Ahora instala herramientas de ProjectDiscovery:	13
go install -v github.com/projectdiscovery/httpx/cmd/httpx@latest	13
go install -v github.com/projectdiscovery/nuclei/v3/cmd/nuclei@latest	13
go install -v github.com/projectdiscovery/subfinder/v2/cmd/subfinder@latest	13
Añadimos Go al PATH:	14
echo 'export PATH=\$PATH:~/go/bin' >> ~/.bashrc	14

source ~/.bashrc.....	14
Comprobamos:.....	14
httpx -h.....	14
nuclei -h.....	14
subfinder -h.....	14
ffuf -h.....	14
• Actualizamos templates de nuclei:.....	14
nuclei -update-templates.....	14
• Y ya para el lab:.....	14
ffuf -w ~/SecLists/Discovery/Web-Content/common.txt -u	
http://dev.lab.local:4444/FUZZ.....	14
• Endpoints ocultos:.....	14
ffuf -w ~/SecLists/Discovery/Web-Content/common.txt -u	
http://test.lab.local:4444/FUZZ.....	14
Un 301 significa:.....	15
“el recurso existe pero redirige”.....	15
O sea:.....	15
• el endpoint NO es falso.....	15
• nginx sabe que existe.....	15
• probablemente redirige a /debug/.....	15
Ahora investigamos el endpoint:.....	15
¿Por qué es vulnerabilidad?.....	16
• Buscar backups:.....	16
ffuf -w ~/SecLists/Discovery/Web-Content/raft-medium-files.txt -u	
http://backup.lab.local:4444/FUZZ.....	16
• Hosts vivos:.....	16
cat hosts.txt httpx -title -status-code -tech-detect.....	16
• Vulnerabilidades/misconfigs:.....	16
nuclei -l hosts.txt -tags exposure,misconfig,backup,debug.....	16
Y honestamente, con eso ya tienes media metodología de reconocimiento web real. 16	
Subfinder.....	17
Httpx.....	17

2. Hallazgo.....	19
• Hallazgo 1.....	20
• Hallazgo 2.....	20
• Hallazgo 3.....	20
• Hallazgo 4.....	21
• Hallazgo 5.....	22
• Hallazgo 6:.....	23
• Hallazgo 7: Evidencia: Análisis de cabeceras HTTP.....	26
3. Comparativa de herramientas utilizadas en el laboratorio.....	28
Uso de cada herramienta en el laboratorio.....	29
ffuf.....	29
httpx.....	29
nuclei.....	30
katana.....	30
subfinder.....	31
Conclusiones:.....	31
4. Glosario resumen.....	35

1. Levantar el laboratorio

HERRAMIENTAS:

- **ffuf** → fuzzing de rutas
- **httpx** → comprobar hosts vivos
- **nuclei** → detectar vulnerabilidades/misconfigs
- **subfinder** → subdominios reales
- opcional: **katana**, **waybackurls**, **hakrawler**

Primero ejecutamos el script:

```
chmod +x demolab.sh  
./demolab.sh
```

Significa:

- **chmod** → cambiar permisos de un archivo
- **+x** → añadir permiso de ejecución
- **demolab.sh** → el script

O sea, le estamos diciendo a Linux:

“este archivo se puede ejecutar como programa”

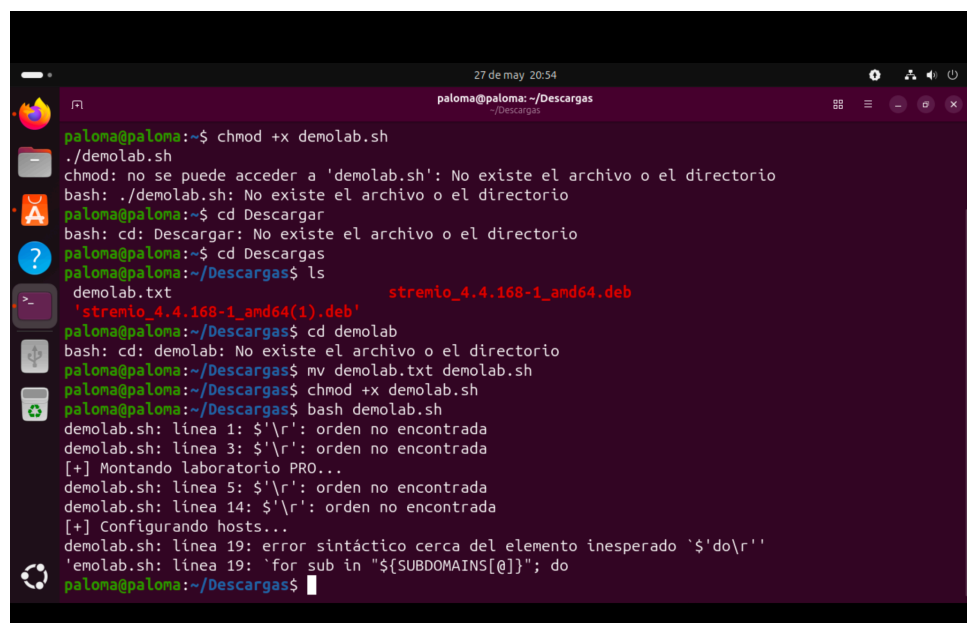
Porque aunque exista el archivo, Linux por seguridad no deja ejecutar scripts automáticamente.

- Luego:

```
./demolab.sh
```

significa:

- **./** →
“ejecuta el
archivo que
está en
ESTA
carpeta”



```
paloma@paloma:~/Descargas$ chmod +x demolab.sh  
paloma@paloma:~/Descargas$ ./demolab.sh  
chmod: no se puede acceder a 'demolab.sh': No existe el archivo o el directorio  
bash: ./demolab.sh: No existe el archivo o el directorio  
paloma@paloma:~/Descargas$ cd Descargas  
bash: cd: Descargas: No existe el archivo o el directorio  
paloma@paloma:~/Descargas$ cd Descargas  
paloma@paloma:~/Descargas$ ls  
demolab.txt      stremio_4.4.168-1_amd64.deb  
stremio_4.4.168-1_amd64(1).deb  
paloma@paloma:~/Descargas$ cd demolab  
bash: cd: demolab: No existe el archivo o el directorio  
paloma@paloma:~/Descargas$ mv demolab.txt demolab.sh  
paloma@paloma:~/Descargas$ chmod +x demolab.sh  
paloma@paloma:~/Descargas$ bash demolab.sh  
demolab.sh: línea 1: '$\r': orden no encontrada  
demolab.sh: línea 3: '$\r': orden no encontrada  
[+] Montando laboratorio PRO...  
demolab.sh: línea 5: '$\r': orden no encontrada  
demolab.sh: línea 14: '$\r': orden no encontrada  
[+] Configurando hosts...  
demolab.sh: línea 19: error sintáctico cerca del elemento inesperado '$\do\r'  
'demolab.sh: línea 19: 'for sub in "${SUBDOMAINS[@]}"; do  
paloma@paloma:~/Descargas$
```

- `demo1ab.sh` → el script

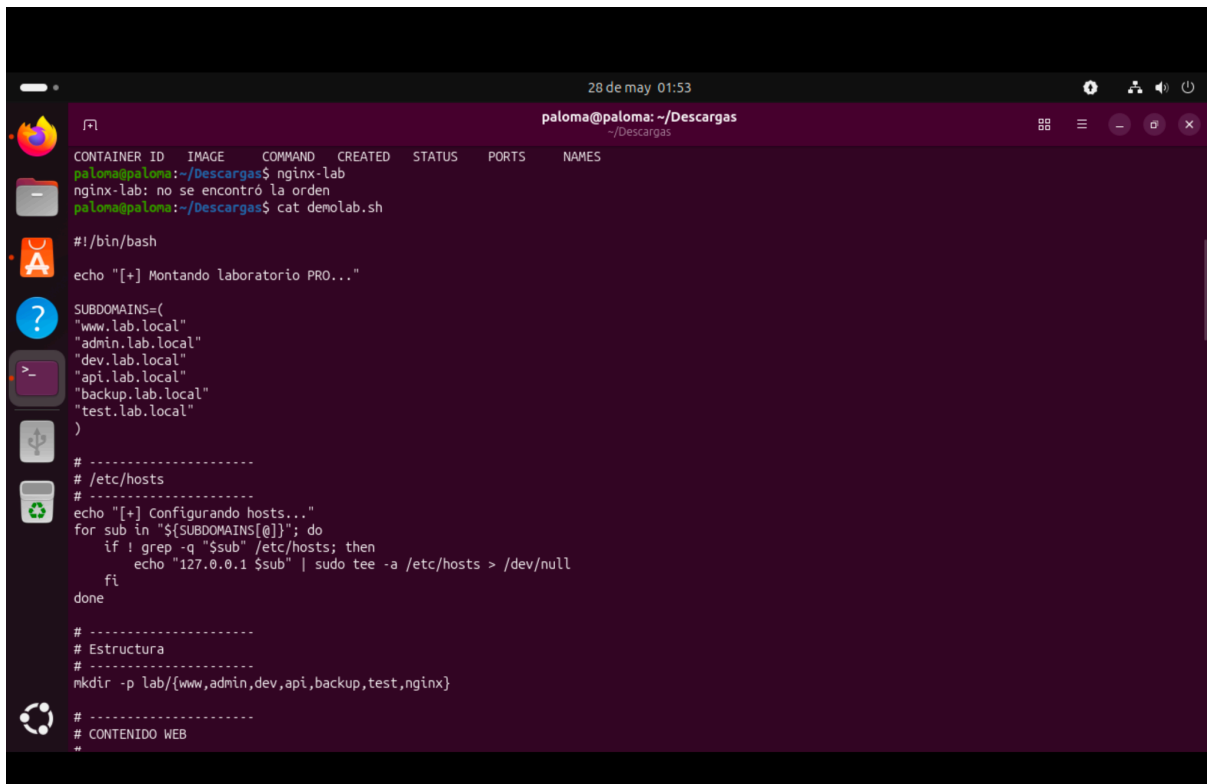
Es como decir:

“lanzamos este script desde aquí mismo”

Sin el `./`, Bash buscaría el programa en rutas del sistema tipo `/usr/bin` y no en tu carpeta actual.

Luego comprobamos que el contenedor está vivo:

`docker ps`



The screenshot shows a terminal window titled "paloma@paloma: ~/Descargas" with a timestamp of "28 de may 01:53". The terminal displays the following commands and output:

```
paloma@paloma:~/Descargas$ nginx-lab
nginx-lab: no se encontró la orden
paloma@paloma:~/Descargas$ cat demo1ab.sh

#!/bin/bash

echo "[+] Montando laboratorio PRO..."

SUBDOMAINS=(
  "www.lab.local"
  "admin.lab.local"
  "dev.lab.local"
  "api.lab.local"
  "backup.lab.local"
  "test.lab.local"
)

# -----
# /etc/hosts
# -----
echo "[+] Configurando hosts..."
for sub in "${SUBDOMAINS[@]}; do
  if ! grep -q "$sub" /etc/hosts; then
    echo "127.0.0.1 $sub" | sudo tee -a /etc/hosts > /dev/null
  fi
done

# -----
# Estructura
# -----
mkdir -p lab/{www,admin,dev,api,backup,test,nginx}

# -----
# CONTENIDO WEB
#
```

```
28 de may 01:53
paloma@paloma: ~/Descargas
# -----
# CONTENIDO WEB
# -----
# WWW (info leakage)
cat > lab/www/index.html <<EOF
<h1>ACME Corp</h1>
<p>Bienvenido</p>
<!-- TODO: migrate dev.lab.local to production -->
EOF
# ADMIN (credenciales en JS)
cat > lab/admin/index.html <<EOF
<h1>Admin Panel</h1>
<script>
var user="admin";
var password="admin123"; // TODO remove before production
</script>
EOF
# DEV (rutas internas)
cat > lab/dev/index.html <<EOF
<h1>Dev Environment</h1>
<p>Debug enabled</p>
<ul>
<li>/test</li>
<li>/backup</li>
<li>/old</li>
</ul>
EOF
# DEV rutas ocultas
mkdir -p lab/dev/backup
echo "backup file" > lab/dev/backup/db.sql.bak
```

```
28 de may 01:54
paloma@paloma: ~/Descargas
mkdir -p lab/dev/backup
echo "backup file" > lab/dev/backup/db.sql.bak
mkdir -p lab/dev/old
echo "old version" > lab/dev/old/index.old
# API
cat > lab/api/index.json <<EOF
{
  "status":"ok",
  "version":"2.3"
}
EOF
# Endpoint oculto
cat > lab/api/secret.txt <<EOF
API_KEY=12345-SECRET
EOF
# BACKUP server
echo "DATABASE DUMP" > lab/backup/db_backup.zip
# TEST server
cat > lab/test/index.html <<EOF
<h1>Testing</h1>
<p>Debug=true</p>
<p>Try /debug</p>
EOF
mkdir -p lab/test/debug
echo "stack trace error" > lab/test/debug/log.txt
# -----
# NGINX CONFIG
# -----
```

```
28 de may 01:54
paloma@paloma: ~/Descargas

listen 4444;
server_name backup.lab.local;
root /var/www/backup;
autoindex on;
}

server {
    listen 4444;
    server_name test.lab.local;
    root /var/www/test;
}
}
EOF

# -----
# DOCKER
# -----

docker rm -f nginx-lab 2>/dev/null

docker run -d \
    --name nginx-lab \
    -p 4444:4444 \
    -v $(pwd)/lab/nginx/nginx.conf:/etc/nginx/nginx.conf:ro \
    -v $(pwd)/lab/www:/var/www/www \
    -v $(pwd)/lab/admin:/var/www/admin \
    -v $(pwd)/lab/dev:/var/www/dev \
    -v $(pwd)/lab/api:/var/www/api \
    -v $(pwd)/lab/backup:/var/www/backup \
    -v $(pwd)/lab/test:/var/www/test \
    nginx

echo ""
echo "[+] LAB PRO listo"
paloma@paloma:~/Descargas$
```

```
28 de may 01:54
paloma@paloma: ~/Descargas

echo "stack trace error" > lab/test/debug/log.txt

# -----
# NGINX CONFIG
# -----

cat > lab/nginx/nginx.conf <<'EOF'
events {}

http {

    server {
        listen 4444;
        server_name www.lab.local;
        root /var/www/www;
    }

    server {
        listen 4444;
        server_name admin.lab.local;
        root /var/www/admin;
    }

    server {
        listen 4444;
        server_name dev.lab.local;
        root /var/www/dev;
    }

    server {
        listen 4444;
        server_name api.lab.local;

        location / {
            root /var/www/api;
        }
    }
}
```



```
28 de may 01:56
paloma@paloma: ~/Descargas
# DOCKER
# -----
docker rm -f nginx-lab 2>/dev/null

docker run -d \
  --name nginx-lab \
  -p 4444:4444 \
  -v $(pwd)/lab/nginx/nginx.conf:/etc/nginx/nginx.conf:ro \
  -v $(pwd)/lab/www:/var/www/www \
  -v $(pwd)/lab/admin:/var/www/admin \
  -v $(pwd)/lab/dev:/var/www/dev \
  -v $(pwd)/lab/api:/var/www/api \
  -v $(pwd)/lab/backup:/var/www/backup \
  -v $(pwd)/lab/test:/var/www/test \
  nginx

echo ""
echo "[+] LAB PRO listo"
paloma@paloma:~/Descargas$ docker ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
728de5fd7669   nginx                                "/docker-entrypoint..." 6 days ago    Exited (255) 6 hours ago    80/tcp, 0.0.0.0:4444->4444/tcp, [::]:4444
a8ebfd8491e4   monarcappfo-monarcfoapp            "docker-entrypoint.s..." 2 weeks ago    Exited (137) 12 days ago
b4971dae1d71   monarcappfo-stats-service          "docker-stats-entryp..." 2 weeks ago    Exited (137) 12 days ago
fc0219d7da2f   postgres:15                         "docker-entrypoint.s..." 2 weeks ago    Exited (0) 12 days ago
15925ea597a8   mariadb:10.11                      "docker-entrypoint.s..." 2 weeks ago    Exited (0) 12 days ago
55d7adf4c2a6   sj26/mailcatcher:latest            "mailcatcher --foreg..." 2 weeks ago    Exited (0) 12 days ago
paloma@paloma:~/Descargas$
```

```
28 de may 01:59
paloma@paloma: ~/Descargas
172.17.0.1 - - [21/May/2026:18:46:46 +0000] "GET /old/ HTTP/1.1" 403 153 "-" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:150.0) Gecko/20100101 Firefox/150.0"
2026/05/21 18:46:46 [error] 22#22: *214 directory index of "/var/www/dev/old/" is forbidden, client: 172.17.0.1, server: dev.lab.local, request: "GET /old/ HTTP/1.1", host: "dev.lab.local:4444"
172.17.0.1 - - [21/May/2026:18:47:24 +0000] "GET / HTTP/1.1" 200 53 "-" "curl/8.14.1"
paloma@paloma:~/Descargas$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
58deea49ef1: Pull complete
c3bdf82c34d1: Download complete
Digest: sha256:0e760fd7bc48ba8041e7c6db999bb40bfca580b4be580ac75d32c4e29d202ce1
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (arm64v8)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
paloma@paloma:~/Descargas$
```

Y verificamos que responde:

curl http://127.0.0.1:4444

O en navegador:

<http://www.lab.local:4444>

```
28 de may 01:59
paloma@paloma: ~/Descargas
~/Descargas
172.17.0.1 - - [21/May/2026:18:46:46 +0000] "GET /old/ HTTP/1.1" 403 153 "-" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:150.0) Gecko/20100101 Firefox/150.0"
2026/05/21 18:46:46 [error] 22#22: *214 directory index of "/var/www/dev/old/" is forbidden, client: 172.17.0.1, server: dev.lab.local, request: "GET /old/ HTTP/1.1", host: "dev.lab.local:4444"
172.17.0.1 - - [21/May/2026:18:47:24 +0000] "GET / HTTP/1.1" 200 53 "-" "curl/8.14.1"
paloma@paloma:~/Descargas$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
58dee6a49ef1: Pull complete
c3bdf82c34d1: Download complete
Digest: sha256:0e760fd9bc40ba8041e7c6db999bb40bfca508b4be580ac75d32c4e29d202ce1
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

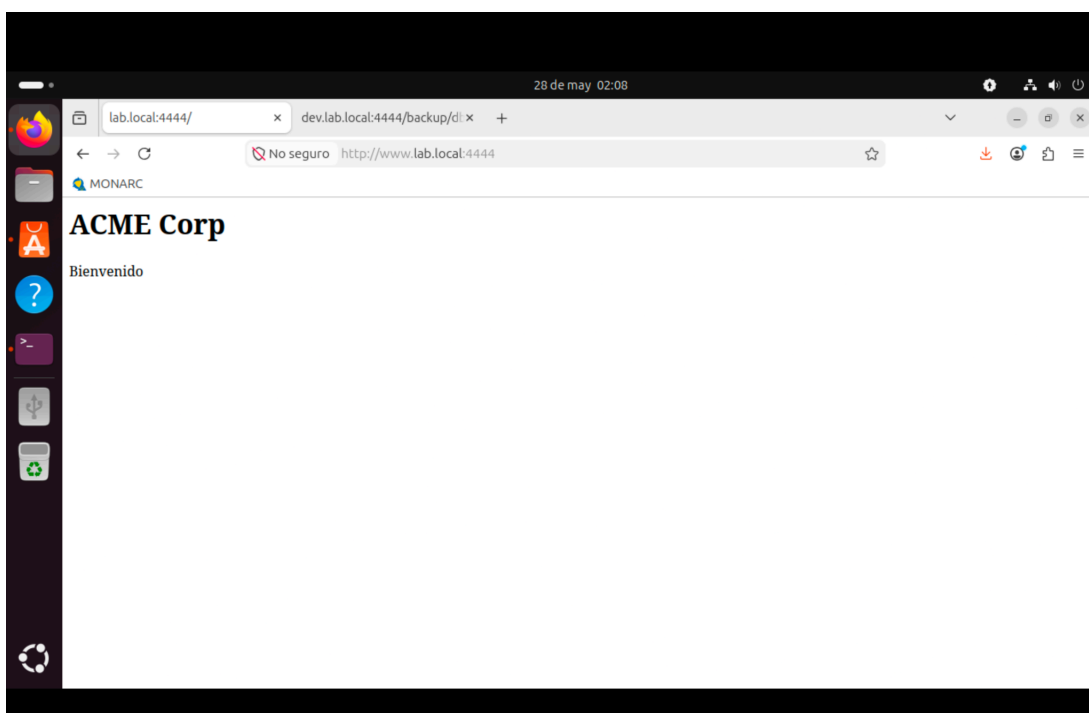
To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (arm64v8)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

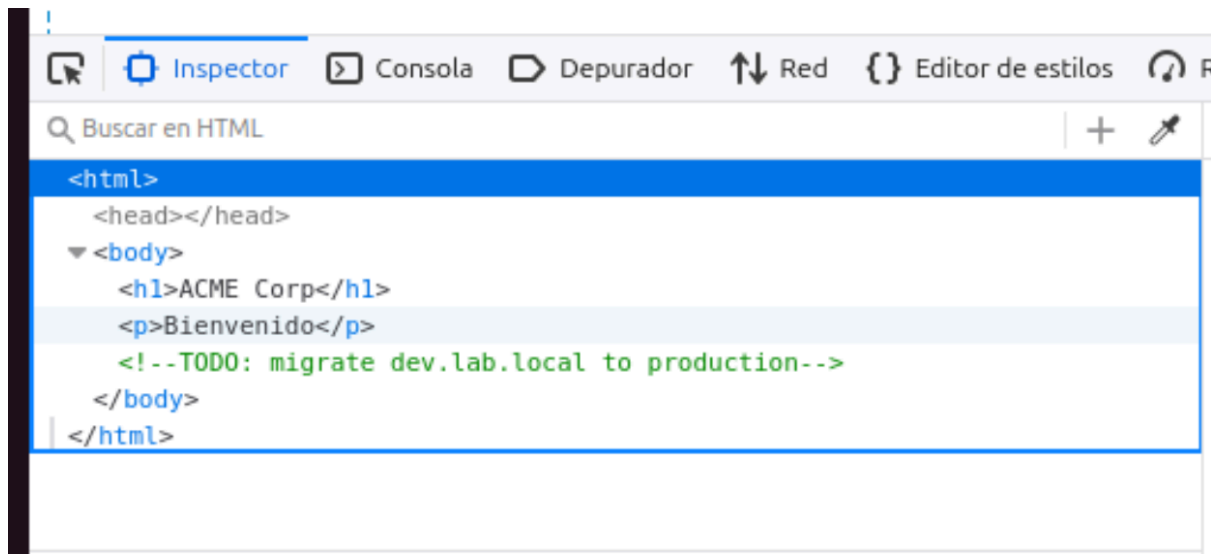
Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
paloma@paloma:~/Descargas$
```

```
nglnx - Lab
paloma@paloma:~/Descargas$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS    PORTS                               NAMES
728de5fd7669   nginx    "/docker-entrypoint...." 6 days ago Up 7 seconds    80/tcp, 0.0.0.0:4444->4444/tcp, [::]:4444->4444/tcp    nginx-lab
paloma@paloma:~/Descargas$
```



`<!-- TODO: migrate dev.lab.local to production -->`



**Eso ya es un hallazgo de información sensible.
Porque te está revelando:**

- existe un entorno *dev*
- probablemente separado de producción
- y puede tener menos seguridad
- debug activado
- backups
- versiones antiguas
- APIs internas

Ahora pensamos como atacante...

Tu siguiente pregunta natural sería:

“¿Qué más hay en dev.lab.local?”

Así que ahora volvemos a enumerar rutas sobre DEV.

Pensemos:

- ya sabemos usar *ffuf*
- ya sabes que *FUZZ* sustituye palabras del diccionario
- ya conoces una wordlist de *Web-Content*

Y tenemos que apuntar a:

http://dev.lab.local:4444/FUZZ

A partir de ahí:

- *mirar qué devuelve 200*
- *qué devuelve 301*
- *qué devuelve 403*
- *qué nombres parecen interesantes*

Y cuando encontremos algo raro tipo:

- *backups*
- *old*
- *debug*
- *logs*
- *zip*
- *sql*

PARA DESCUBRIR RUTAS

28 de may 16:31

paloma@paloma: ~/Descargas

```

:: Matcher                : Response status: 200-299,301,302,307,401,403,405,500

```

```

backpup                  [Status: 301, Size: 169, Words: 5, Lines: 8, Duration: 1ms]
index.html               [Status: 200, Size: 103, Words: 3, Lines: 8, Duration: 2ms]
old                     [Status: 301, Size: 169, Words: 5, Lines: 8, Duration: 0ms]
:: Progress: [4751/4751] :: Job [1/1] :: 0 req/sec :: Duration: [0:00:00] :: Errors: 0 ::
paloma@paloma:~/Descargas$ ffuf -u http://dev.lab.local:4444/FUZZ \
-w ~/SecLists/Discovery/Web-Content/common.txt

```



v2.1.0-dev

```

:: Method                : GET
:: URL                   : http://dev.lab.local:4444/FUZZ
:: Wordlist               : FUZZ: /home/paloma/SecLists/Discovery/Web-Content/common.txt
:: Follow redirects      : false
:: Calibration           : false
:: Timeout               : 10
:: Threads               : 40
:: Matcher               : Response status: 200-299,301,302,307,401,403,405,500

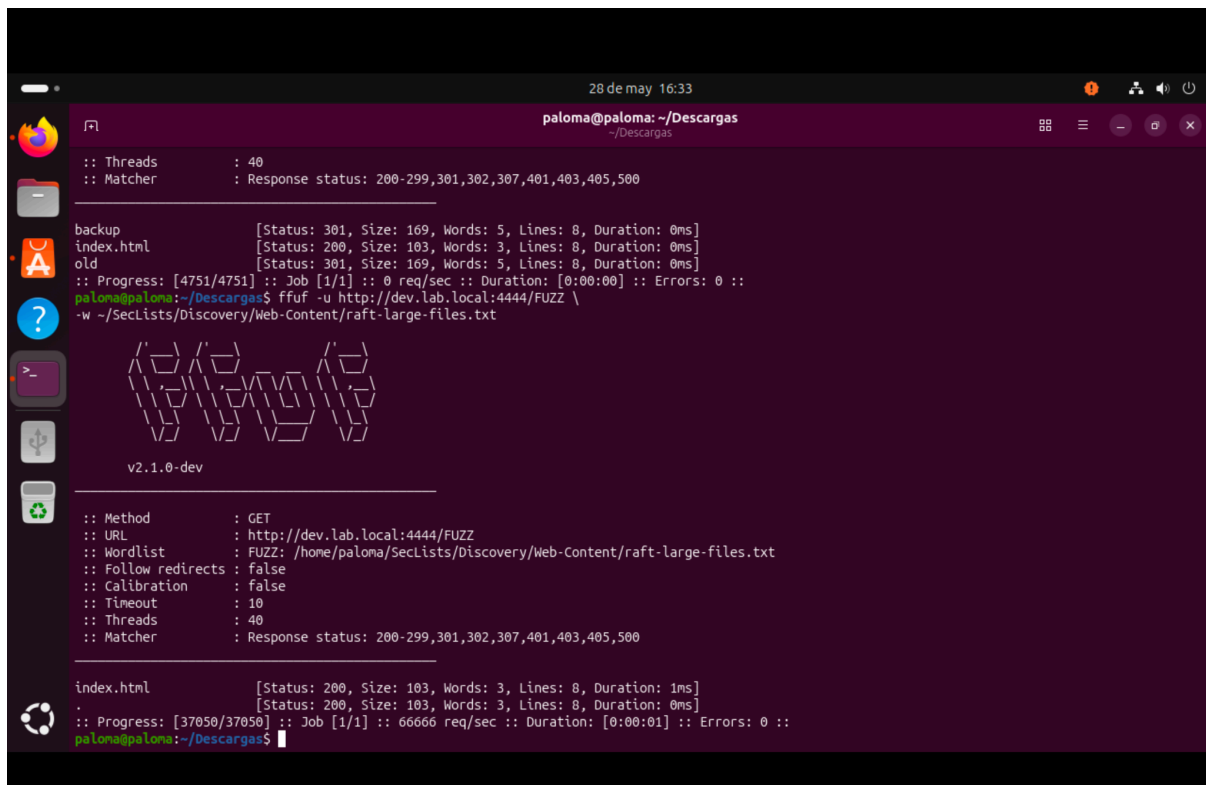
```

```

backpup                  [Status: 301, Size: 169, Words: 5, Lines: 8, Duration: 0ms]
index.html               [Status: 200, Size: 103, Words: 3, Lines: 8, Duration: 0ms]
old                     [Status: 301, Size: 169, Words: 5, Lines: 8, Duration: 0ms]
:: Progress: [4751/4751] :: Job [1/1] :: 0 req/sec :: Duration: [0:00:00] :: Errors: 0 ::
paloma@paloma:~/Descargas$

```

Buscar archivos backup



The screenshot shows a terminal window titled 'paloma@paloma: ~/Descargas' with a timestamp of '28 de may 16:33'. It displays the output of an ffuf command. The terminal shows the progress of a fuzzing job, with a list of discovered files: 'backup', 'index.html', and 'old'. The status for each file is shown, including HTTP status, size, words, lines, and duration. The terminal also shows the command used: 'ffuf -u http://dev.lab.local:4444/FUZZ -w ~/SecLists/Discovery/Web-Content/raft-large-files.txt'. The output shows that the files were discovered with a status of 301 and a size of 169 bytes.

```
28 de may 16:33
paloma@paloma: ~/Descargas

:: Threads      : 40
:: Matcher      : Response status: 200-299,301,302,307,401,403,405,500

backup      [Status: 301, Size: 169, Words: 5, Lines: 8, Duration: 0ms]
index.html  [Status: 200, Size: 103, Words: 3, Lines: 8, Duration: 0ms]
old         [Status: 301, Size: 169, Words: 5, Lines: 8, Duration: 0ms]
:: Progress: [4751/4751] :: Job [1/1] :: 0 req/sec :: Duration: [0:00:00] :: Errors: 0 ::
paloma@paloma:~/Descargas$ ffuf -u http://dev.lab.local:4444/FUZZ \
-w ~/SecLists/Discovery/Web-Content/raft-large-files.txt

v2.1.0-dev

:: Method      : GET
:: URL         : http://dev.lab.local:4444/FUZZ
:: Wordlist     : FUZZ: /home/paloma/SecLists/Discovery/Web-Content/raft-large-files.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Response status: 200-299,301,302,307,401,403,405,500

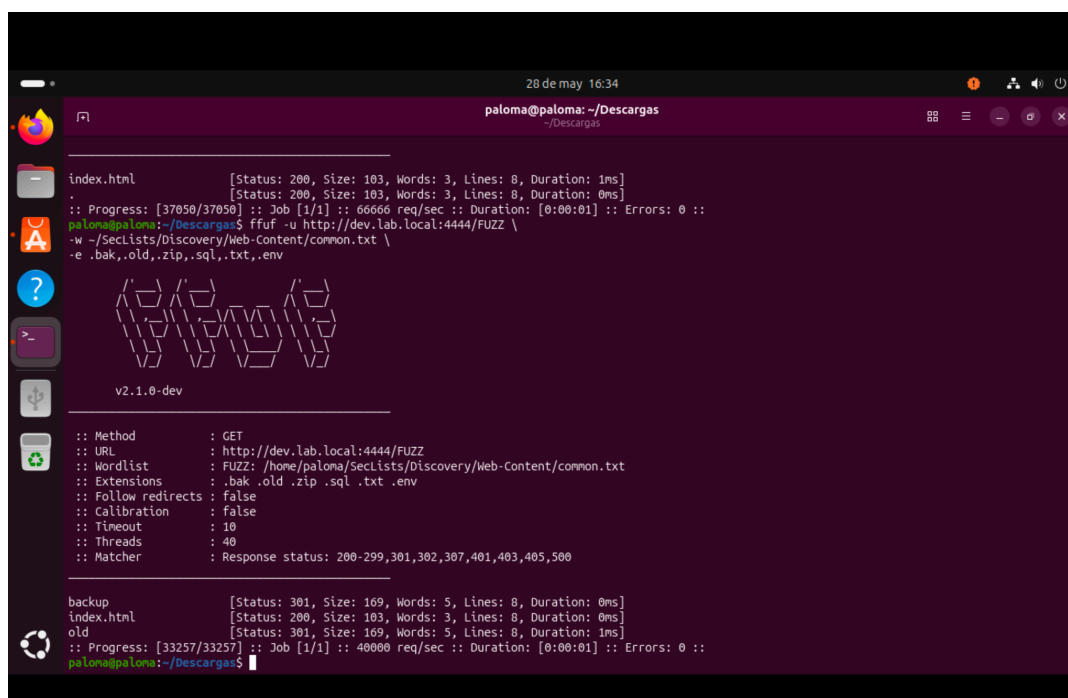
index.html    [Status: 200, Size: 103, Words: 3, Lines: 8, Duration: 1ms]
.             [Status: 200, Size: 103, Words: 3, Lines: 8, Duration: 0ms]
:: Progress: [37050/37050] :: Job [1/1] :: 66666 req/sec :: Duration: [0:00:01] :: Errors: 0 ::
paloma@paloma:~/Descargas$
```

ffuf -u http://dev.lab.local:4444/FUZZ \

-w ~/SecLists/Discovery/Web-Content/common.txt \

-e .bak,.old,.zip,.sql,.txt,.env

Buscar extensiones típicas



The screenshot shows a terminal window titled 'paloma@paloma: ~/Descargas' with a timestamp of '28 de may 16:34'. It displays the output of an ffuf command. The terminal shows the progress of a fuzzing job, with a list of discovered files: 'index.html', 'backup', 'index.html', and 'old'. The status for each file is shown, including HTTP status, size, words, lines, and duration. The terminal also shows the command used: 'ffuf -u http://dev.lab.local:4444/FUZZ -w ~/SecLists/Discovery/Web-Content/common.txt -e .bak,.old,.zip,.sql,.txt,.env'. The output shows that the files were discovered with a status of 301 and a size of 169 bytes.

```
28 de may 16:34
paloma@paloma: ~/Descargas

index.html    [Status: 200, Size: 103, Words: 3, Lines: 8, Duration: 1ms]
.             [Status: 200, Size: 103, Words: 3, Lines: 8, Duration: 0ms]
:: Progress: [37050/37050] :: Job [1/1] :: 66666 req/sec :: Duration: [0:00:01] :: Errors: 0 ::
paloma@paloma:~/Descargas$ ffuf -u http://dev.lab.local:4444/FUZZ \
-w ~/SecLists/Discovery/Web-Content/common.txt \
-e .bak,.old,.zip,.sql,.txt,.env

v2.1.0-dev

:: Method      : GET
:: URL         : http://dev.lab.local:4444/FUZZ
:: Wordlist     : FUZZ: /home/paloma/SecLists/Discovery/Web-Content/common.txt
:: Extensions  : .bak .old .zip .sql .txt .env
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Response status: 200-299,301,302,307,401,403,405,500

backup      [Status: 301, Size: 169, Words: 5, Lines: 8, Duration: 0ms]
index.html  [Status: 200, Size: 103, Words: 3, Lines: 8, Duration: 0ms]
old         [Status: 301, Size: 169, Words: 5, Lines: 8, Duration: 1ms]
:: Progress: [33257/33257] :: Job [1/1] :: 40000 req/sec :: Duration: [0:00:01] :: Errors: 0 ::
paloma@paloma:~/Descargas$
```

```
ffuf -u http://dev.lab.local:4444/FUZZ \  
  
-w ~/SecLists/Discovery/Web-Content/common.txt \  
  
-e .bak,.old,.zip,.sql,.txt,.env
```

NUCLEI → buscar vulnerabilidades conocidas

Primero instalamos templates si no están:

```
:: Progress: [33257/33257] :: Job [1/1] :: 40000 req/sec :: D  
paloma@paloma:~/Descargas$ nuclei -update-templates  
  
nuclei  
v3.8.0  
projectdiscovery.io  
  
[INF] No new updates found for nuclei templates  
paloma@paloma:~/Descargas$
```

nuclei -update-templates

Escaneo básico

nuclei -u <http://www.lab.local:4444>

```
paloma@paloma:~/Descargas$ nuclei -u http://www.lab.local:4444  
  
nuclei  
v3.8.0  
projectdiscovery.io  
  
[INF] Found 2 templates with runtime error (use -validate flag for further examination)  
[INF] Current nuclei version: v3.8.0 (latest)  
[INF] Current nuclei-templates version: v10.4.3 (latest)  
[INF] New templates added in latest release: 103  
[INF] Templates loaded for current scan: 10194  
[INF] Executing 10178 signed templates from projectdiscovery/nuclei-templates  
[WARN] Loading 16 unsigned templates for scan. Use with caution.  
[INF] Targets loaded for current scan: 1  
[INF] Templates clustered: 2292 (Reduced 2162 Requests)  
[INF] Using Interactsh Server: oast.site  
  
waf-detect:nginxgeneric [http] [info] http://www.lab.local:4444  
dameng-detect [javascript] [info] www.lab.local:4444  
sonarjs-detect [javascript] [info] www.lab.local:4444 ["Enterprise: unknown"]  
http-missing-security-headers:cross-origin-embedder-policy [http] [info] http://www.lab.local:4444  
http-missing-security-headers:permissions-policy [http] [info] http://www.lab.local:4444  
http-missing-security-headers:x-frame-options [http] [info] http://www.lab.local:4444  
http-missing-security-headers:x-content-type-options [http] [info] http://www.lab.local:4444  
http-missing-security-headers:clear-site-data [http] [info] http://www.lab.local:4444  
http-missing-security-headers:cross-origin-opener-policy [http] [info] http://www.lab.local:4444  
http-missing-security-headers:cross-origin-resource-policy [http] [info] http://www.lab.local:4444  
http-missing-security-headers:missing-content-type [http] [info] http://www.lab.local:4444  
http-missing-security-headers:strict-transport-security [http] [info] http://www.lab.local:4444  
http-missing-security-headers:content-security-policy [http] [info] https://www.lab.local:4444  
http-missing-security-headers:x-permitted-cross-domain-policies [http] [info] http://www.lab.local:4444  
http-missing-security-headers:referrer-policy [http] [info] http://www.lab.local:4444  
nginx-version [http] [info] http://www.lab.local:4444 ["nginx/1.31.0"]  
tech-detect:nginx [http] [info] http://www.lab.local:4444  
[INF] Skipped www.lab.local:4040 from target list as found unresponsive permanently: cause="port closed or filtered" address=www.lab.local:4040 chain="connection refused; got err while executing https  
://www.lab.local:4040/jobs/?i">script=alert(document.domain)</script>  
[INF] Skipped [www.lab.local:4444]:5814 from target list as found unresponsive permanently: Get "https://www.lab.local:4444:5814/autopass": cause="no address found for host"  
[INFO] Skipped www.lab.local:4444 from target list as found unresponsive permanently: cause="no address found for host" chain="not err while executing https://www.lab.local:4444:5814/autopass"
```

```
[nghttp-version] [http] [info] http://www.lab.local:4444 ["nghttp/1.31.0"]
[tech-detect:nghttp] [http] [info] http://www.lab.local:4444
[INF] Skipped www.lab.local:4040 from target list as found unresponsive permanently: cause="port closed or filtered" address=www.lab.local:4040 chain="connection refused; got err while executing https
://www.lab.local:4040/jobs/?\"><script>alert(document.domain)</script>"
[INF] Skipped [www.lab.local:4444]:5814 from target list as found unresponsive permanently: Get "https://www.lab.local:4444:5814/autopass": cause="no address found for host"
[INF] Skipped www.lab.local:4444 from target list as found unresponsive permanently: cause="no address found for host" chain="got err while executing https://www.lab.local:4444:5814/autopass"
[caa-fingerprint] [dns] [info] www.lab.local
[INF] Scan completed in 1m. 18 matches found.
paloma@paloma:~/Descargas$
```

Sí, para el lab te renta tener el stack típico instalado:

- **ffuf** → fuzzing de rutas
- **httpx** → comprobar hosts vivos
- **nuclei** → detectar vulnerabilidades/misconfigs
- **subfinder** → subdominios reales
- opcional: **katana**, **waybackurls**, **hakrawler**

En Ubuntu hacemos esto para instalar las herramientas nuclei, https, subfinder.:

```
sudo apt update
```

```
sudo apt install ffuf golang git -y
```

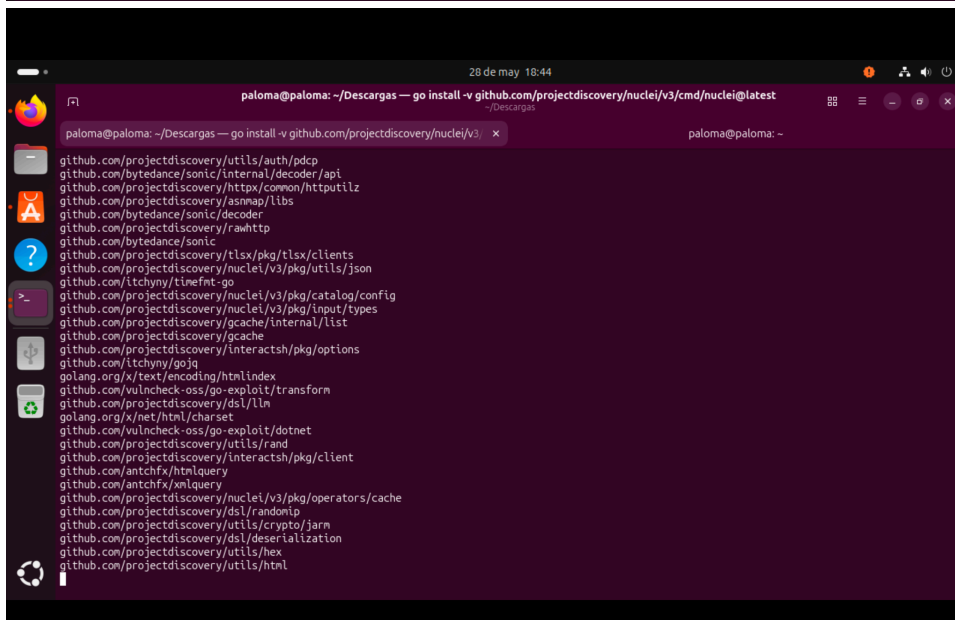
Ahora instala herramientas de ProjectDiscovery:

```
go install -v github.com/projectdiscovery/httpx/cmd/httpx@latest
```

```
go install -v github.com/projectdiscovery/nuclei/v3/cmd/nuclei@latest
```

```
go install -v github.com/projectdiscovery/subfinder/v2/cmd/subfinder@latest
```

```
Configurando golang.alm04 (2.1.24~2) ...
paloma@paloma:~/Descargas$ go install -v github.com/projectdiscovery/httpx/cmd/httpx@latest
go install -v github.com/projectdiscovery/nuclei/v3/cmd/nuclei@latest
go install -v github.com/projectdiscovery/subfinder/v2/cmd/subfinder@latest
go: github.com/projectdiscovery/httpx@v1.9.0 requires go >= 1.25.7; switching to go1.25.10
go: github.com/projectdiscovery/nuclei/v3@v3.8.0 requires go >= 1.25.7; switching to go1.25.10
```



Añadimos Go al PATH:

```
echo 'export PATH=$PATH:~/go/bin' >> ~/.bashrc
```

```
source ~/.bashrc
```

Comprobamos:

```
httpx -h
```

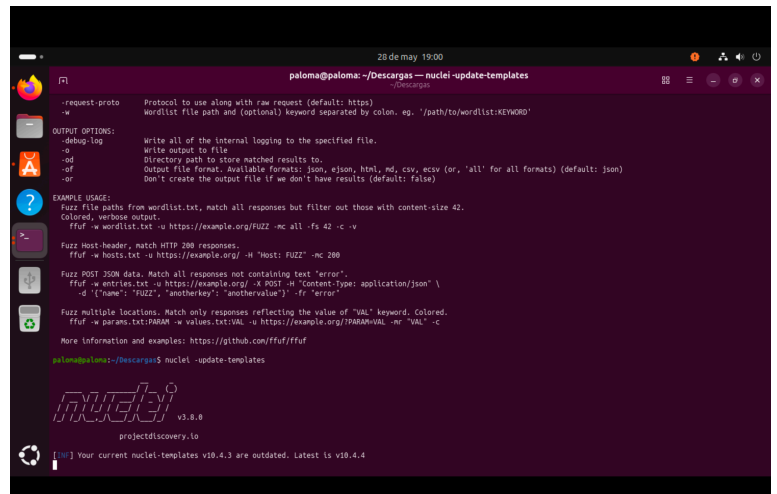
```
nuclei -h
```

```
subfinder -h
```

```
ffuf -h
```

- Actualizamos templates de nuclei:

```
nuclei -update-templates
```

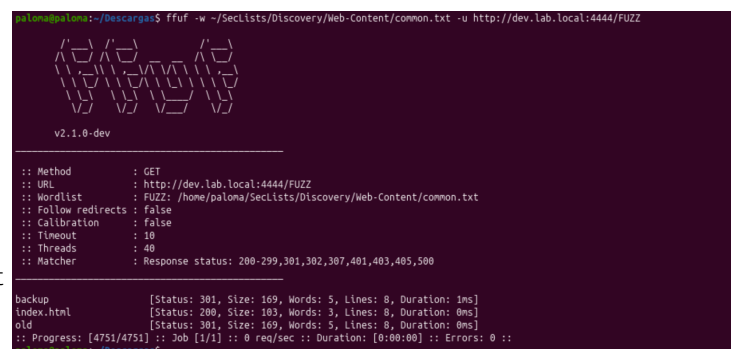


- Y ya para el lab:

```
ffuf -w ~/SecLists/Discovery/Web-Content/common.txt -u  
http://dev.lab.local:4444/FUZZ
```

- Endpoints ocultos:

```
ffuf -w  
~/SecLists/Discovery/Web-Content/common.txt  
-u http://test.lab.local:4444/FUZZ
```



Endpoint	Código	Qué significa
/debug	301	Existe y redirige
/index.html	200	Página accesible correctamente

Un **301** significa:

“el recurso existe pero redirige”.

O sea:

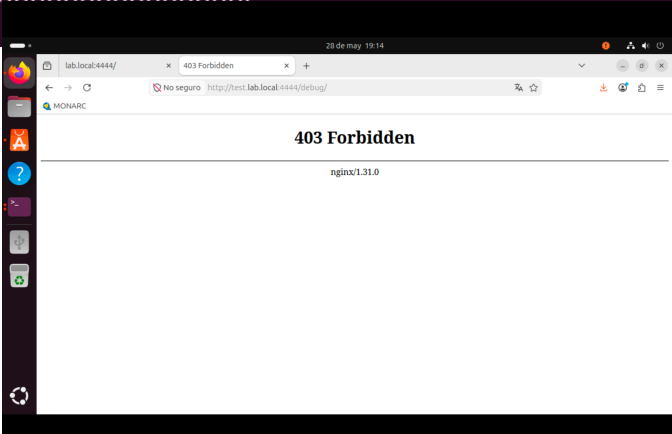
- *el endpoint NO es falso*
- *nginx sabe que existe*
- *probablemente redirige a **/debug/***

Eso ya es interesante porque:

- *“debug” suele contener logs*
- *errores*
- *trazas*
- *configuración*
- *información sensible*

Ahora investigamos el endpoint:

```
paloma@paloma:~/Descargas$ curl -I http://test.lab.local:4444/debug
HTTP/1.1 301 Moved Permanently
Server: nginx/1.31.0
Date: Thu, 28 May 2026 17:13:57 GMT
Content-Type: text/html
Content-Length: 169
Location: http://test.lab.local:4444/debug/
Connection: keep-alive
```



¿Por qué es vulnerabilidad?

Porque un entorno de **debug expuesto**:

- *revela estructura interna*
- *ayuda al atacante*
- *puede mostrar rutas*
- *tecnologías*
- *nombres de archivos*
- *mensajes de error*

Eso facilita ataques posteriores !!!

- **Buscar backups:**

```
ffuf -w ~/SecLists/Discovery/Web-Content/raft-medium-files.txt -u  
http://backup.lab.local:4444/FUZZ
```

- **Hosts vivos:**

```
cat hosts.txt | httpx -title -status-code -tech-detect
```

- **Vulnerabilidades/misconfigs:**

```
nuclei -l hosts.txt -tags exposure,misconfig,backup,debug
```

Y honestamente, con eso ya tienes media metodología de reconocimiento web real.

Usamos subfinder ahora para encontrar subdominios que puedan aportar algo más a nuestra auditoría.

```
paloma@paloma:~/Descargas$ ~/go/bin/subfinder -d lab.local -silent
ciscoasa.lab.local
trclabcas2.lab.local
beheer-linux.lab.local
db-1.lab.local
niobe.lab.local
tank.lab.local
lynxsrvc.sfj.lab.local
clinta.lab.local
mirage.lab.local
zion.lab.local
labserver.lab.local
ora-xe-18c.lab.local
construct.lab.local
trinity.ilo.lab.local
m1.lab.local
labms001.lab.local
server01.lab.local
morpheus.ilo.lab.local
morpheus.lab.local
web-2.lab.local
m0.lab.local
architect.lab.local
autodiscover.lab.local
lync.lab.local
trclabcas1.lab.local
lynxedge.sfj.lab.local
exc2007.lab.local
db-2.lab.local
dozer.lab.local
tank.ilo.lab.local
trinity.lab.local
paloma@paloma:~/Descargas$
```

Esto subfinder lo está sacando de fuentes OSINT, no necesariamente significa que estén en mi laboratorio.

Subfinder

Dice:

"He visto referencias a estos nombres en Internet"

Httpx

Dice:

"Este host responde realmente"

Veamos cuáles están vivos...

[illegible]

```
cat subdominios.txt | ~/go/bin/httpx -title -status-code
```

Sirve para **comprobar cuáles de los subdominios encontrados están realmente activos**. Primero, `cat subdominios.txt` lee la lista de dominios guardada en el fichero. El símbolo `|` (pipe) envía esa lista directamente a `httpx`. Después, `httpx` intenta conectarse a cada dominio y muestra información útil: `-status-code` indica el código HTTP que devuelve el servidor (200, 403, 404, etc.) y `-title` muestra el título de la página web. En resumen, **Subfinder encuentra posibles dominios y Httpx verifica cuáles existen y responden realmente**. 😎

Se utilizó **Subfinder** para obtener posibles subdominios del dominio **lab.local**. Posteriormente se validaron con **Httpx** mediante el comando `cat subdominios.txt | httpx -title -status-code`. No se obtuvieron respuestas válidas, por lo que los subdominios encontrados no estaban accesibles o no resolvían correctamente.

2. Hallazgos

Hallazgo 1

URL:

http://test.lab.local:4444/debug/

Tipo:

Exposición de directorio de debug

Impacto:

El directorio de debug expone información interna del entorno de pruebas. Un atacante podría obtener logs, errores o trazas internas que faciliten el reconocimiento del sistema y futuros ataques.

Cómo se descubrió:

Se utilizó la herramienta ffuf junto con la wordlist common.txt de SecLists para realizar fuzzing de directorios sobre test.lab.local.

Comando utilizado:

```
bash ffuf -w ~/SecLists/Discovery/Web-Content/common.txt -u  
http://test.lab.local:4444/FUZZ
```

Resultado obtenido:

debug [Status: 301]

Posteriormente se accedió manualmente al endpoint /debug/.

Mitigación:

Deshabilitar directorios de debug en entornos de producción, restringir el acceso mediante autenticación y eliminar archivos internos innecesarios accesibles públicamente.

Hallazgo 2

URL:

http://admin.lab.local:4444/

Tipo:

Credenciales hardcodeadas en código JavaScript

Impacto:

Las credenciales expuestas en el código fuente pueden ser utilizadas por un atacante para acceder al panel de administración. Este tipo de exposición compromete la confidencialidad y puede derivar en accesos no autorizados.

Cómo se descubrió:

Se accedió al subdominio `admin.lab.local` descubierto durante la enumeración y posteriormente se revisó el código fuente de la página.

Comando utilizado:

```
curl http://admin.lab.local:4444
```

También puede verificarse desde el navegador usando “Ver código fuente”.

Resultado encontrado:

```
var user="admin";  
var password="admin123";
```

Mitigación:

Nunca almacenar credenciales en código frontend o JavaScript accesible públicamente. Utilizar variables de entorno y mecanismos seguros de autenticación.

Hallazgo 3

URL:

```
http://dev.lab.local:4444/backup/db.sql.bak
```

Tipo:

Exposición de archivo de backup

Impacto:

La exposición de backups puede permitir a un atacante acceder a bases de datos completas, credenciales, información sensible o estructuras internas de la aplicación.

Cómo se descubrió:

Se utilizó `ffuf` con una wordlist de SecLists para descubrir rutas ocultas en el entorno de desarrollo.

Comando utilizado:

```
ffuf -w ~/SecLists/Discovery/Web-Content/common.txt -u  
http://dev.lab.local:4444/FUZZ
```

Posteriormente se identificó el directorio `/backup/` y el archivo `db.sql.bak`.

Mitigación:

Eliminar backups accesibles públicamente, restringir el acceso mediante autenticación y almacenar copias de seguridad fuera del directorio web.

Hallazgo 4

URL:

`http://api.lab.local:4444/secret.txt`

Tipo:

Exposición de clave API

Impacto:

La clave API expuesta puede ser utilizada por terceros no autorizados para interactuar con la API, comprometiendo servicios internos y acceso a datos.

Cómo se descubrió:

Se realizó fuzzing de archivos y directorios utilizando `ffuf` sobre el subdominio `api.lab.local`.

Comando utilizado:

```
ffuf -w ~/SecLists/Discovery/Web-Content/common.txt -u  
http://api.lab.local:4444/FUZZ
```

Resultado encontrado:

`API_KEY=12345-SECRET`

Mitigación:

No almacenar claves API en archivos públicos accesibles desde web. Utilizar variables de entorno y sistemas seguros de gestión de secretos.

LOS INDICADOS ARRIBA SON LOS MÁS RELEVANTES.

Herramienta	Hallazgo principal
FFUF	/debug, /backup, secret.txt, backups expuestos
HTTPX	Hosts activos, 403 Forbidden, detección de Nginx
Subfinder	Enumeración de subdominios
Katana	Crawling de enlaces
Nuclei	Cabeceras de seguridad ausentes, fingerprinting de tecnologías

Y un Hallazgo 5 basado en Nuclei:

Hallazgo 5

URL:

<http://dev.lab.local:4444>

```
paloma@paloma: ~/Descargas$ ~/go/bin/nuclei -u http://dev.lab.local:4444

nuclei v3.8.0
projectdiscovery.io

[WRN] Found 1 templates with runtime error (use -validate flag for further examination)
[INF] Current nuclei version: v3.8.0 (latest)
[INF] Current nuclei-templates version: v10.4.4 (latest)
[INF] New templates added in latest release: 179
[INF] Templates loaded for current scan: 10365
[INF] Executing 10349 signed templates from projectdiscovery/nuclei-templates
[WRN] Loading 17 unsigned templates for scan. Use with caution.
[INF] Targets loaded for current scan: 1
[INF] Templates clustered: 2376 (Reduced 2244 Requests)
[INF] Using Interactsh Server: oast.site
[waf-detect:nginxgeneric] [http] [info] http://dev.lab.local:4444
```

```
3 de jun 00:32
paloma@paloma: ~/Descargas
~/Descargas

[INF] Current nuclei-templates version: v10.4.4 (latest)
[INF] New templates added in latest release: 179
[INF] Templates loaded for current scan: 10365
[INF] Executing 10349 signed templates from projectdiscovery/nuclei-templates
[WRN] Loading 17 unsigned templates for scan. Use with caution.
[INF] Targets loaded for current scan: 1
[INF] Templates clustered: 2376 (Reduced 2244 Requests)
[INF] Using Interactsh Server: oast.site
[waf-detect:nginxgeneric] [http] [info] http://dev.lab.local:4444
[snmpv3-detect] [javascript] [info] dev.lab.local:4444 ["Enterprise: unknown"]
[tech-detect:nginx] [http] [info] http://dev.lab.local:4444
[http-missing-security-headers:strict-transport-security] [http] [info] http://dev.lab.local:4444
[http-missing-security-headers:content-security-policy] [http] [info] http://dev.lab.local:4444
[http-missing-security-headers:referrer-policy] [http] [info] http://dev.lab.local:4444
[http-missing-security-headers:cross-origin-embedder-policy] [http] [info] http://dev.lab.local:4444
[http-missing-security-headers:cross-origin-resource-policy] [http] [info] http://dev.lab.local:4444
[http-missing-security-headers:permissions-policy] [http] [info] http://dev.lab.local:4444
[http-missing-security-headers:x-frame-options] [http] [info] http://dev.lab.local:4444
[http-missing-security-headers:x-content-type-options] [http] [info] http://dev.lab.local:4444
[http-missing-security-headers:permitted-cross-domain-policies] [http] [info] http://dev.lab.local:4444
[http-missing-security-headers:cross-origin-opener-policy] [http] [info] http://dev.lab.local:4444
[nginx-version] [http] [info] http://dev.lab.local:4444 ["nginx/1.31.0"]
[INF] Skipped dev.lab.local:4444:5814 from target list as found unresponsive permanently: Get "https://dev.lab.local:4444:5814/autopass": cause="no address found for host"
[INF] Skipped dev.lab.local:4444 from target list as found unresponsive permanently: cause="no address found for host" chain="got err while executing https://dev.lab.local:4444:5814/autopass"
[INF] Skipped dev.lab.local:4040 from target list as found unresponsive permanently: cause="port closed or filtered" address=dev.lab.local:4040 chain="connection refused; got err while executing https://dev.lab.local:4040/jobs/?i="">script=alert(document.domain)</script>"
[caa-fingerprint] [dns] [info] dev.lab.local
[INF] Scan completed in 1m. 17 matches found.
paloma@paloma: ~/Descargas$
```


Tipo:

Ausencia de cabeceras de seguridad HTTP

Impacto:

La aplicación no implementa varias cabeceras de seguridad recomendadas como `Content-Security-Policy`, `X-Frame-Options` o `X-Content-Type-Options`. Esto puede facilitar ataques de clickjacking, inyección de contenido o problemas relacionados con el aislamiento entre orígenes.

Cómo se descubrió:

Se utilizó la herramienta Nuclei para analizar automáticamente configuraciones inseguras del servidor web.

Comando utilizado:

```
~/go/bin/nuclei -u http://dev.lab.local:4444
```

Resultado encontrado:

http-missing-security-headers:strict-transport-security

http-missing-security-headers:content-security-policy

http-missing-security-headers:x-frame-options

http-missing-security-headers:x-content-type-options

Mitigación:

Configurar las cabeceras de seguridad recomendadas en Nginx y aplicar una política de endurecimiento (hardening) del servidor web.

Justo esta captura es una evidencia perfecta de **Subfinder**. Si yo fuera estudiante, la documentaría así:

Hallazgo 6:

URL/Dominio analizado:

`lab.local`

Tipo:

Enumeración de subdominios

Impacto:

La identificación de subdominios permite ampliar la superficie de ataque y descubrir servicios adicionales que podrían no ser visibles desde el dominio principal.

Cómo se descubrió:

Se utilizó la herramienta Subfinder para realizar una enumeración pasiva de subdominios asociados al dominio `lab.local`.

Comando utilizado:

```
~/go/bin/subfinder -d lab.local -silent
```

Resultado obtenido:

niobe.lab.local

beheer-linux.lab.local

construct.lab.local

dozer.lab.local

trinity.lab.local

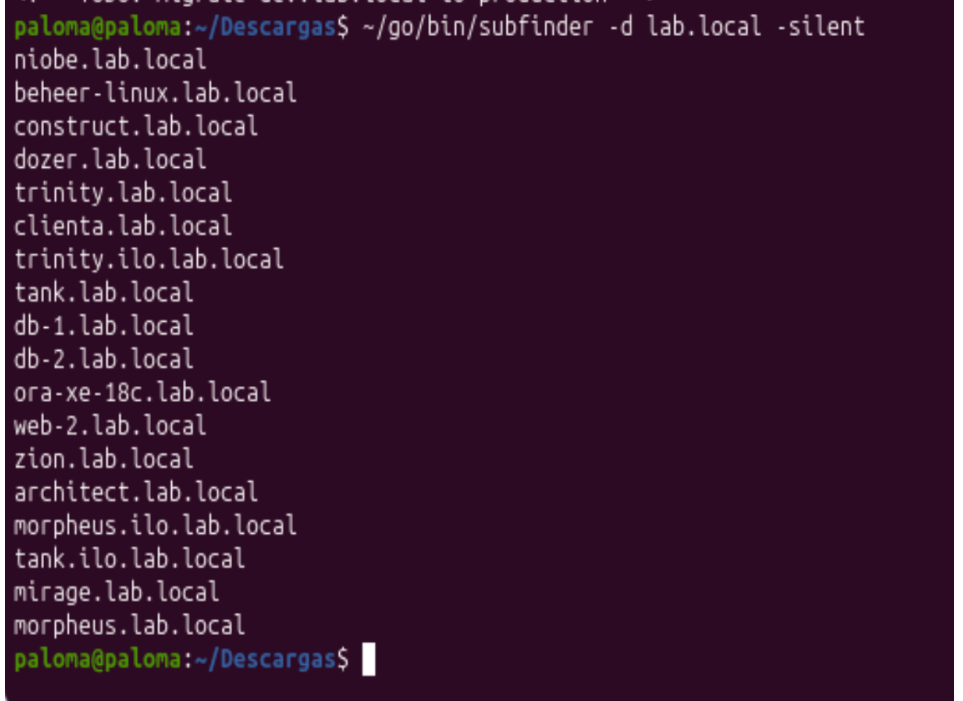
cliente.lab.local

db-1.lab.local

db-2.lab.local

ora-xe-18c.lab.local

web-2.lab.local



```
paloma@paloma:~/Descargas$ ~/go/bin/subfinder -d lab.local -silent
niobe.lab.local
beheer-linux.lab.local
construct.lab.local
dozer.lab.local
trinity.lab.local
clienta.lab.local
trinity.ilo.lab.local
tank.lab.local
db-1.lab.local
db-2.lab.local
ora-xe-18c.lab.local
web-2.lab.local
zion.lab.local
architect.lab.local
morpheus.ilo.lab.local
tank.ilo.lab.local
mirage.lab.local
morpheus.lab.local
paloma@paloma:~/Descargas$
```

Mitigación:

Mantener un inventario actualizado de activos expuestos y eliminar registros DNS o subdominios que ya no sean necesarios.

Subfinder encontró numerosos subdominios relacionados con bases de datos (`db-1`, `db-2`, `ora-xe-18c`), gestión (`cliente`, `construct`) y servidores (`web-2`, `beheer-linux`). Posteriormente sería necesario validar cuáles están realmente existen o estan “vivas” mediante HTTPX.

```
~/go/bin/subfinder -d lab.local -silent > subdominios.txt
```

```
cat subdominios.txt | ~/go/bin/httpx -status-code -title
```

```
paloma@paloma:~/Descargas$ ~/go/bin/subfinder -d lab.local -silent > subdominios.txt

cat subdominios.txt | ~/go/bin/httpx -status-code -title

projectdiscovery.io

[INF] Current httpx version v1.9.0 (latest)
[WRN] UI Dashboard is disabled, Use -dashboard option to enable
paloma@paloma:~/Descargas$
```

Subfinder identificó múltiples subdominios potenciales asociados a **lab.local**, pero HTTPX no detectó ningún servicio HTTP accesible en ellos. Esto indica que dichos subdominios no exponen aplicaciones web, no son resolubles desde el entorno del laboratorio o requieren configuraciones adicionales para ser alcanzados.

La evidencia es muy buena porque demuestra que **Subfinder encontró nombres**, pero esos nombres **no existen en el DNS del laboratorio**.

```
[WRN] UI Dashboard is disabled, Use -dashboard option to enable
paloma@paloma:~/Descargas$ cat subdominios.txt
niobe.lab.local
ora-xe-18c.lab.local
db-1.lab.local
morpheus.ilo.lab.local
trinity.ilo.lab.local
mirage.lab.local
morpheus.lab.local
trinity.lab.local
beheer-linux.lab.local
dozer.lab.local
tank.lab.local
web-2.lab.local
zion.lab.local
architect.lab.local
clinta.lab.local
db-2.lab.local
construct.lab.local
tank.ilo.lab.local
paloma@paloma:~/Descargas$ host morpheus.lab.local
host db-1.lab.local
host cliente.lab.local
Host morpheus.lab.local not found: 5(REFUSED)
Host db-1.lab.local not found: 5(REFUSED)
Host cliente.lab.local not found: 5(REFUSED)
paloma@paloma:~/Descargas$ ping -c 1 morpheus.lab.local
ping: morpheus.lab.local: Fallo temporal en la resolución del nombre
paloma@paloma:~/Descargas$
```

Resultado de validación de subdominios

Tras la enumeración realizada con Subfinder, se procedió a validar varios de los subdominios encontrados mediante las herramientas **host**, **ping** y **httpx**.

Comandos utilizados:

`host morpheus.lab.local` Consulta el DNS para comprobar si el subdominio `morpheus.lab.local` existe y tiene una dirección IP asociada.

`host db-1.lab.local` Verifica si el subdominio `db-1.lab.local` puede resolverse mediante DNS.

`host cliente.lab.local` Comprueba si `cliente.lab.local` está registrado y es resoluble en DNS.

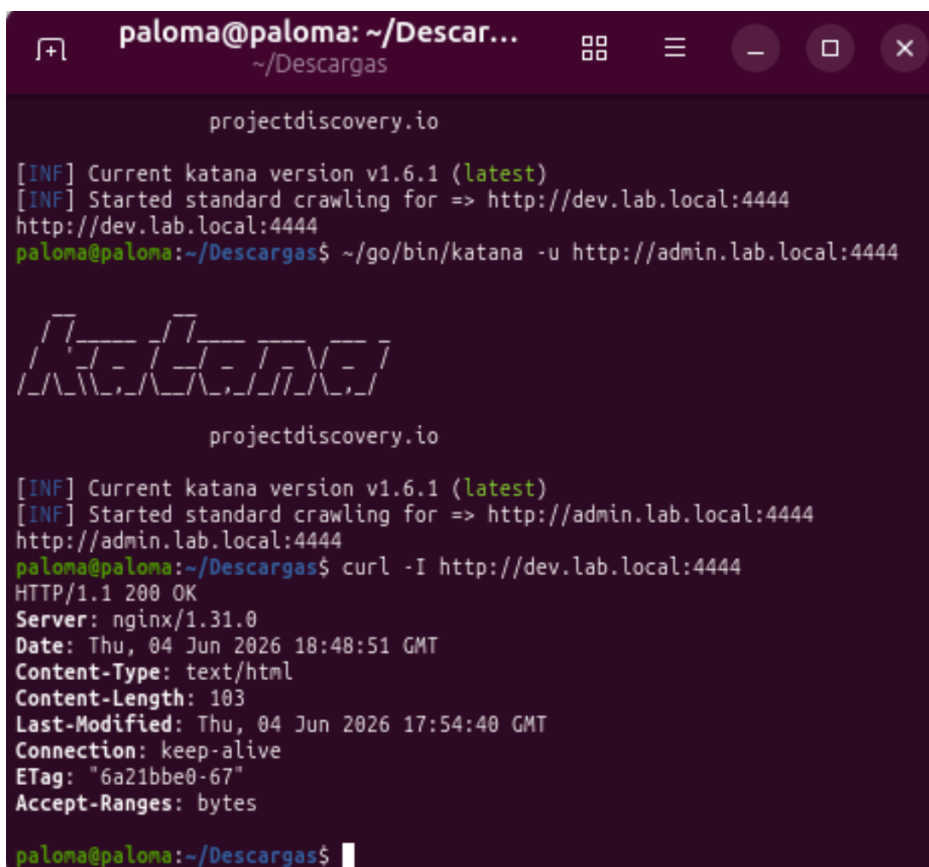
`ping -c 1 morpheus.lab.local` Envía un único paquete ICMP al host para comprobar si responde y si su nombre puede resolverse correctamente.

Conclusión:

Los subdominios identificados por Subfinder no pudieron resolverse mediante DNS ni respondieron a peticiones HTTP. Esto indica que se trata de referencias obtenidas por fuentes OSINT o registros históricos y no de activos accesibles dentro del laboratorio.

Perfecto 😎. Esta evidencia encaja muy bien con el hallazgo de Nuclei.

Hallazgo 7: Evidencia: Análisis de cabeceras HTTP



```
paloma@paloma: ~/Descar...  
~/Descargas  
projectdiscovery.io  
[INF] Current katana version v1.6.1 (latest)  
[INF] Started standard crawling for => http://dev.lab.local:4444  
http://dev.lab.local:4444  
paloma@paloma:~/Descargas$ ~/go/bin/katana -u http://admin.lab.local:4444  
  
A-_-_-_-_-  
projectdiscovery.io  
[INF] Current katana version v1.6.1 (latest)  
[INF] Started standard crawling for => http://admin.lab.local:4444  
http://admin.lab.local:4444  
paloma@paloma:~/Descargas$ curl -I http://dev.lab.local:4444  
HTTP/1.1 200 OK  
Server: nginx/1.31.0  
Date: Thu, 04 Jun 2026 18:48:51 GMT  
Content-Type: text/html  
Content-Length: 103  
Last-Modified: Thu, 04 Jun 2026 17:54:40 GMT  
Connection: keep-alive  
ETag: "6a21bbe0-67"  
Accept-Ranges: bytes  
paloma@paloma:~/Descargas$
```

URL:

<http://dev.lab.local:4444>

Tipo:

Análisis de configuración HTTP

Cómo se descubrió:

Se utilizaron las cabeceras HTTP devueltas por el servidor para verificar la configuración de seguridad.

Comando utilizado:

```
curl -I http://dev.lab.local:4444
```

Resultado obtenido:

HTTP/1.1 200 OK

Server: nginx/1.31.0

Content-Type: text/html

Observaciones:

No aparecen cabeceras de seguridad como:

Strict-Transport-Security

Content-Security-Policy

X-Frame-Options

X-Content-Type-Options

Lo que coincide con los resultados obtenidos previamente mediante Nuclei.

Mitigación:

Configurar las cabeceras de seguridad recomendadas en Nginx para reducir riesgos relacionados con clickjacking, carga de contenido inseguro y otros ataques basados en navegador.

3. Comparativa de herramientas utilizadas en el laboratorio

Herramienta	Función principal	Qué descubre	Ventajas	Limitaciones
ffuf	Fuzzing de rutas y archivos	Directorios ocultos, backups, debug, endpoints	Muy rápido y potente para enumeración	Necesita wordlists
httpx	Verificación de hosts/endpoints	Hosts vivos, títulos, tecnologías, status codes	Ideal para filtrar objetivos válidos	No detecta vulnerabilidades
nuclei	Detección automática de vulnerabilidades	Misconfigurations, exposición, CVEs, backups	Automatiza comprobaciones de seguridad	Depende de templates
katana	Crawling de aplicaciones web	Enlaces internos y rutas navegables	Descubre rutas automáticamente	No encuentra rutas no enlazadas
subfinder	Enumeración de subdominios	Subdominios públicos	Muy útil en dominios reales	En laboratorios locales tiene poca utilidad

Uso de cada herramienta en el laboratorio

ffuf

Se utilizó para descubrir directorios y endpoints ocultos.

Comando utilizado:

```
ffuf -w ~/SecLists/Discovery/Web-Content/common.txt -u  
http://test.lab.local:4444/FUZZ
```

Hallazgos:

- `/debug`
- `/backup`
- `db.sql.bak`
- `secret.txt`

httpx

Se utilizó para verificar qué hosts respondían correctamente y obtener información básica.

Archivo hosts.txt:

```
http://www.lab.local:4444  
http://admin.lab.local:4444  
http://dev.lab.local:4444  
http://api.lab.local:4444  
http://backup.lab.local:4444  
http://test.lab.local:4444
```

Comando utilizado:

```
cat hosts.txt | httpx -title -status-code -tech-detect
```

Información obtenida:

- Status codes
- Tecnologías detectadas
- Títulos HTML
- Hosts activos

nuclei

Se utilizó para buscar vulnerabilidades y malas configuraciones automáticamente.

Comando utilizado:

```
nuclei -l hosts.txt -tags exposure,misconfig,debug,backup
```

Hallazgos:

- Directorios debug expuestos
- Archivos sensibles
- Posibles exposiciones de backup

katana

Se utilizó para **crawling**¹ y descubrimiento automático de rutas.

Comando utilizado:

```
katana -u http://dev.lab.local:4444
```

Rutas detectadas:

- **/backup**
- **/old**
- **/test**

subfinder

Se probó para enumeración de subdominios.

Comando utilizado:

```
subfinder -d lab.local
```

Resultado:

No se obtuvieron resultados relevantes debido a que **lab.local** es un entorno local y no un dominio público.

Conclusiones:

Durante el laboratorio se utilizaron diferentes herramientas orientadas a reconocimiento web y detección de vulnerabilidades.

- **ffuf** fue especialmente útil para descubrir rutas ocultas mediante fuzzing.

¹ *Crawling es un término que engloba el concepto de rastreo, justo la acción que el principal buscador mundial, Google, realiza constantemente en las webs que forman parte de la red.*

- **httpx** permitió validar rápidamente qué hosts estaban activos y obtener información HTTP básica.
- **nuclei** automatizó la detección de exposiciones y malas configuraciones.
- **katana** ayudó a descubrir rutas navegables internas mediante crawling.
- **subfinder** no resultó útil en este laboratorio concreto por tratarse de un dominio local no público.

La combinación de todas estas herramientas permitió realizar una metodología básica de reconocimiento y enumeración similar a la utilizada en auditorías web reales.

4. Glosario resumen:

Herramienta	¿Para qué sirve?	Uso típico en auditoría
ffuf	Fuzzing de rutas, archivos y subdominios	Descubrir <code>/admin</code> , <code>/backup</code> , <code>dev.lab.local</code> , etc
nuclei	Detectar vulnerabilidades y malas configuraciones mediante templates	Buscar exposición de backups, debug, paneles, CVEs
httpx	Comprobar hosts vivos y obtener información HTTP	Ver status code, títulos, tecnologías, redirects
subfinder	Enumeración pasiva de subdominios	Descubrir subdominios públicos reales
katana	Crawling automático de webs	Descubrir rutas siguiendo enlaces internos
curl	Realizar peticiones HTTP manuales	Ver respuestas del servidor y probar endpoints
SecLists	Colección de diccionarios para fuzzing	Wordlists para rutas, DNS, usuarios y archivos
hashcat	Crackeo de hashes	Recuperar contraseñas desde MD5/SHA1/etc

hashid	Identificar tipo de hash	Saber si un hash es MD5, SHA1, SHA256...
cmatrix	Animación estilo Matrix	Estética hacker 🐱
fastfetch	Mostrar info del sistema de forma visual	Ver arquitectura, RAM, shell, kernel, etc
btop	Monitor visual de sistema	Ver CPU, RAM, procesos y red
cava	Visualizador de audio en terminal	Visual aesthetic terminal
docker	Contenedores aislados	Levantar laboratorios y servicios vulnerables
nginx	Servidor web	Simular entornos web y virtual hosts

GLOSARIO MOOMIN

Reconocimiento Web – Herramientas utilizadas



1. SUBFINDER (Snufkin)

Busca subdominios usando información pública de Internet.

★ Sirve para descubrir posibles objetivos.



EJEMPLO:

```
ejemplo.com
↓
api.ejemplo.com
vpn.ejemplo.com
dev.ejemplo.com
mail.ejemplo.com
blog.ejemplo.com
```

¿CÓMO?

Consulta fuentes públicas (OSINT) como certificados, motores, APIs, etc.





2. HTTPX (Moomin)

Comprueba qué dominios están vivos y responden.

★ Sirve para verificar si un host existe realmente.

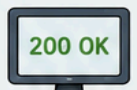


EJEMPLO:

api.ejemplo.com	200 OK
admin.ejemplo.com	403 Forbidden
dev.ejemplo.com	200 OK
blog.ejemplo.com	404 Not Found

¿CÓMO?

Hace peticiones HTTP a cada dominio y muestra el código de estado y el título de la página.





3. FFUF (Little My)

Prueba miles de rutas usando un diccionario (Wordlist).

★ Sirve para encontrar directorios y archivos ocultos.



EJEMPLO:

/admin	200 OK
/backup	200 OK
/debug	200 OK
/old	301 Moved
/config.php	403 Forbidden

¿CÓMO?

Envía muchas peticiones con palabras de un diccionario y encuentra lo que existe.





4. KATANA (Snorkmaiden)

Recorre una web siguiendo enlaces.

★ Sirve para descubrir páginas enlazadas dentro del sitio.



EJEMPLO:

```
Inicio (/)
  ↓      ↓
/productos /contacto
  ↓      ↓
/login     /soporte
```

¿CÓMO?

Extrae enlaces de la web y sigue el rastro para encontrar más páginas.





5. NUCLEI (Hemulen)

Busca vulnerabilidades con plantillas predefinidas.

★ Sirve para detectar problemas de seguridad.



EJEMPLO:

/backup	[!] Backup expuesto
/debug	[!] Debug activado
/login	[!] Posible fuerza bruta
/config.php	[!] Información sensible

¿CÓMO?

Compara respuestas con miles de plantillas de vulnerabilidades conocidas.





6. DICCIONARIOS (Mymble)

Listas de palabras que usan las herramientas.

★ Sirven para adivinar rutas, archivos, subdominios, parámetros, etc.



EJEMPLO DE WORDLIST:

admin	backup	debug	api
login	test	old	dev
config	internal	secure	private
upload	include	config.php	.env

¿CÓMO?

Se usan en herramientas como FFUF para probar muchas combinaciones.



FLUJO COMPLETO DEL RECONOCIMIENTO

